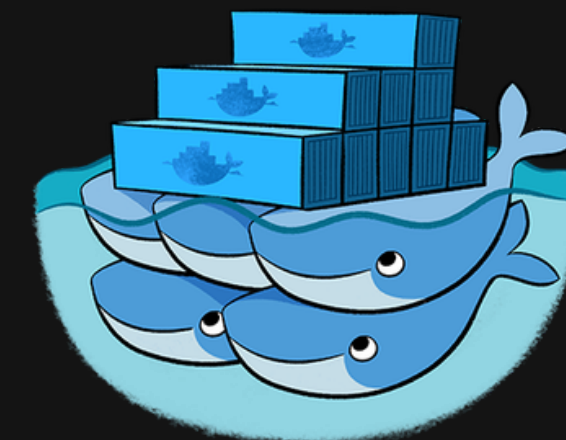


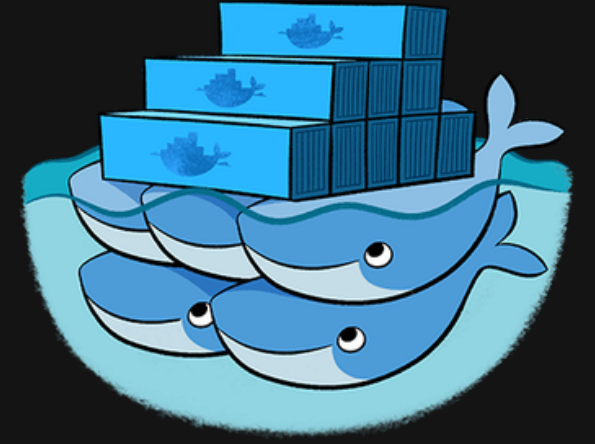


Docker: Contenedores, Imágenes y Orquestación con Compose

Ing. Daniel Reginaldo Dubón Rodríguez

Docker

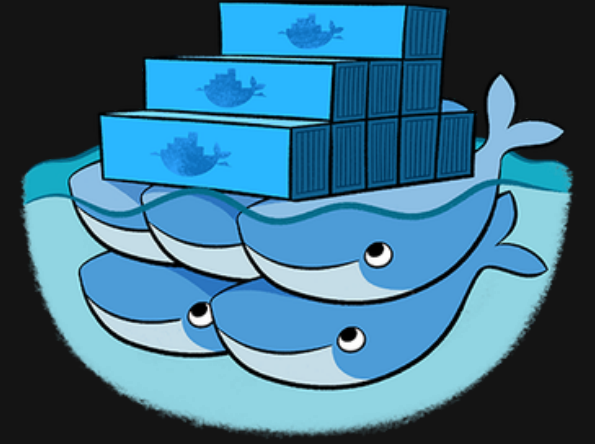
¿Qué es?



Docker es una plataforma para **desarrollar, empaquetar, distribuir y ejecutar** aplicaciones utilizando **contenedores**.

Su objetivo es separar la aplicación de la infraestructura donde se ejecuta, haciendo que el proceso de desarrollo, pruebas y despliegue sea más consistente

Docker



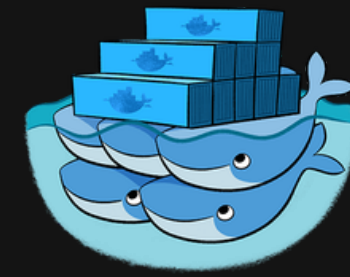
¿Qué pasa si desarrollo una aplicación en mi computadora y al pasarla a otra computadora deja de funcionar?

Las causas pueden ser:

- Diferentes versiones de software.
- Dependencias faltantes.
- Configuraciones diferentes.
- Variables de entorno.
- Diferencias entre sistemas operativos.
- Versiones diferentes de bases de datos.

Docker

¿De dónde surge Docker?



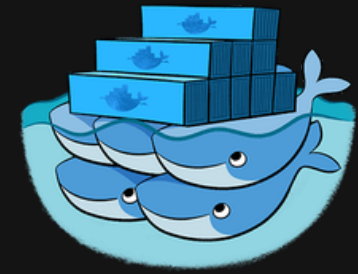
Los contenedores de software tienen antecedentes anteriores a Docker, especialmente en tecnologías de aislamiento disponibles en sistemas Unix/Linux. Docker aprovechó mecanismos del kernel de Linux, como namespaces y cgroups, y los convirtió en una experiencia mucho más accesible para desarrolladores.

Antes de Docker ya existían tecnologías y conceptos relacionados con:

- chroot
- Linux namespaces
- Linux cgroups
- LXC
- Virtualización
- Aislamiento de procesos

Docker

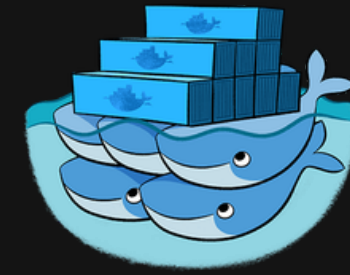
¿De dónde surge Docker?



“Shipping code to the server is hard”

Docker

Su impacto



La principal aportación de Docker fue hacer que trabajar con contenedores fuera mucho más accesible para los desarrolladores.

Docker proporcionó herramientas para:

- Crear imágenes.
- Ejecutar contenedores.
- Compartir imágenes.
- Distribuir aplicaciones.
- Automatizar procesos.
- Administrar redes.
- Administrar almacenamiento.

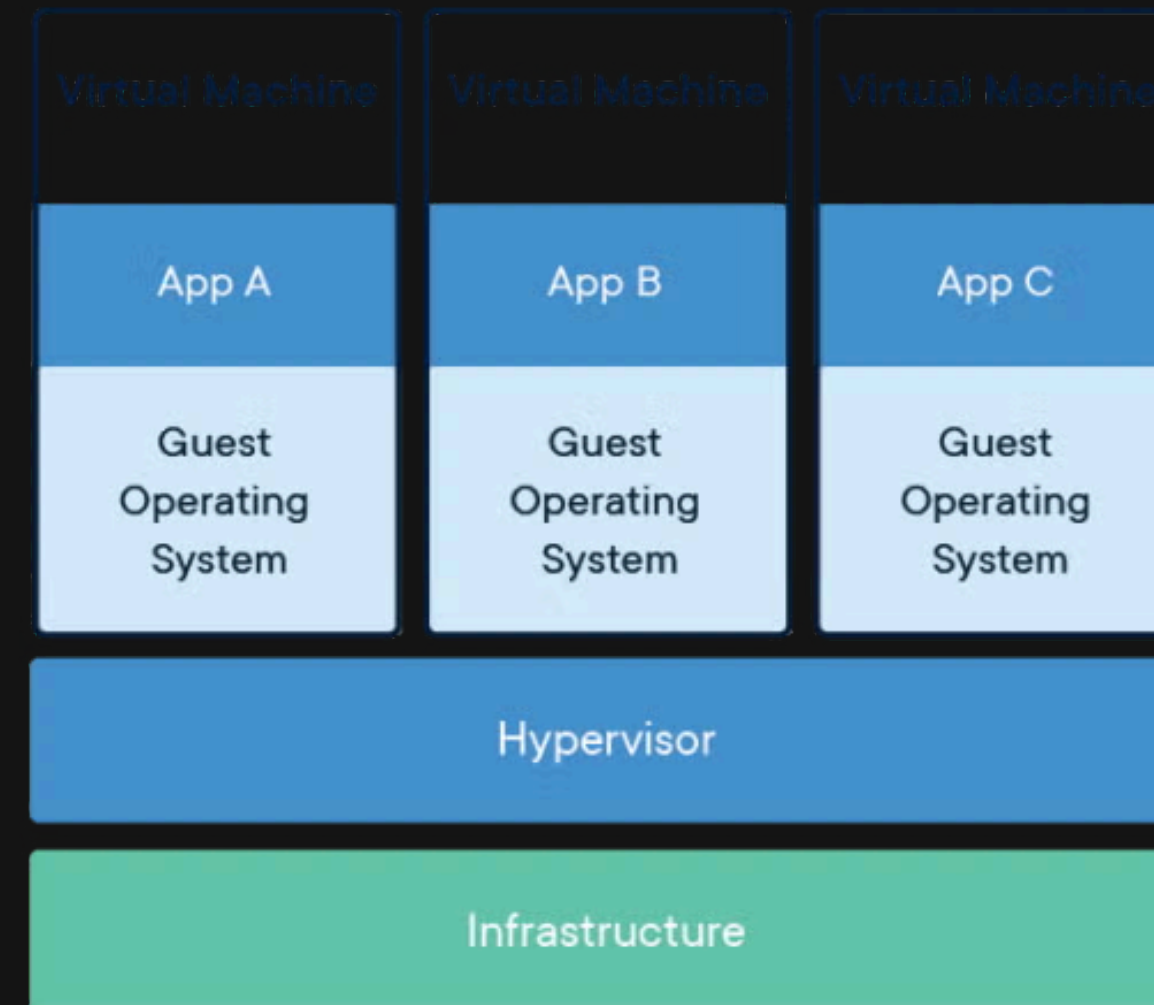
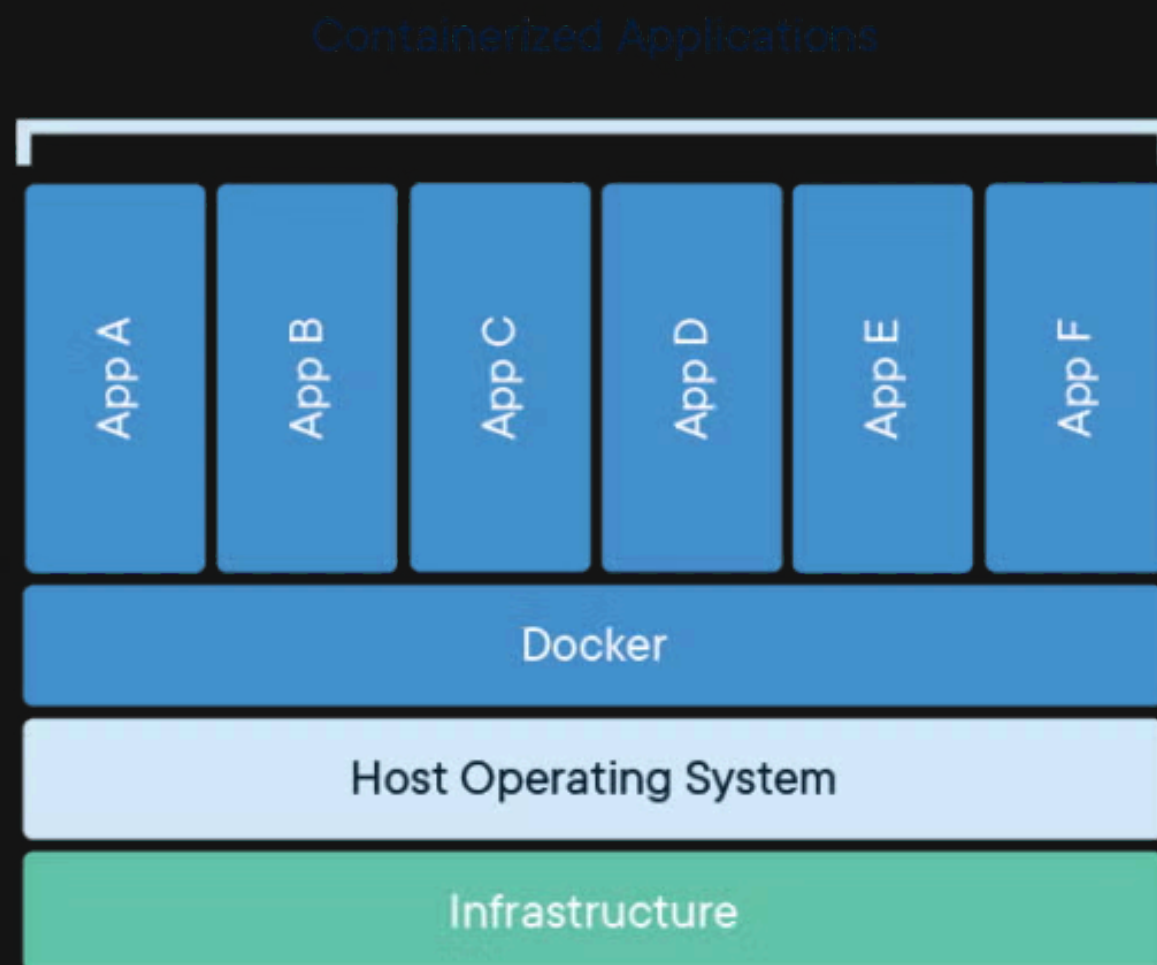
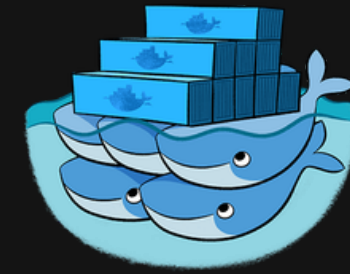
Esto ayudó a convertir los contenedores en una unidad estándar para distribuir y ejecutar aplicaciones.

Virtual Machine vs Docker



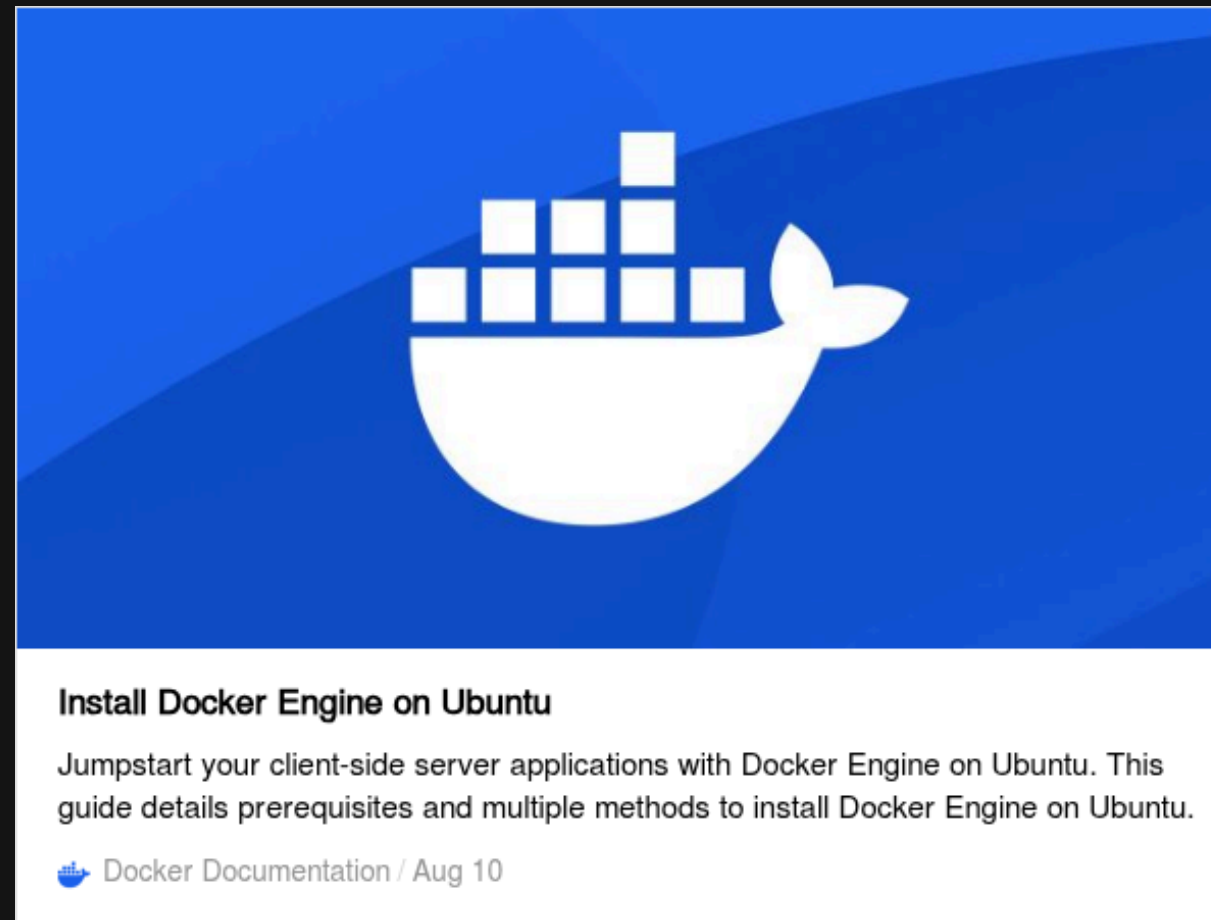
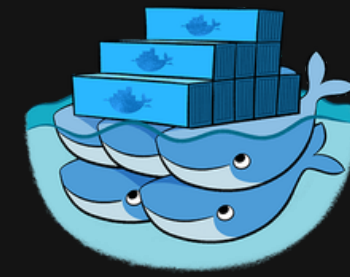
Máquina Virtual (VM)	Docker / Contenedor
Virtualiza una máquina completa.	Aísla una aplicación y sus dependencias.
Incluye un sistema operativo completo.	Comparte el kernel del sistema anfitrión.
Mayor consumo de recursos.	Menor consumo de recursos.
Inicio más lento.	Inicio rápido.
Mayor aislamiento.	Mayor eficiencia y portabilidad.

Virtual Machine vs Docker



Docker

Instalación



Docker

Componentes

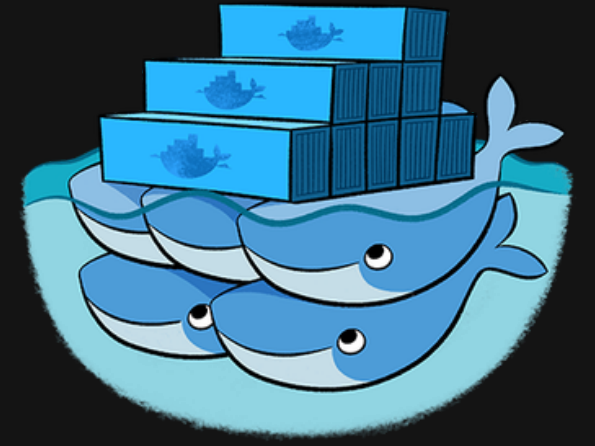


Imagen (Image)

Es una plantilla inmutable que contiene la aplicación, sus dependencias, librerías y configuración necesarias para ejecutarla. A partir de una imagen se pueden crear uno o varios contenedores.

```
> docker images
> docker pull nginx
> docker build -t mi-app .
```

Contenedor (Container)

Es una instancia de una imagen en ejecución. Proporciona un entorno aislado donde la aplicación puede ejecutarse de manera independiente.

```
> docker run nginx docker ps
> docker stop <container>
```

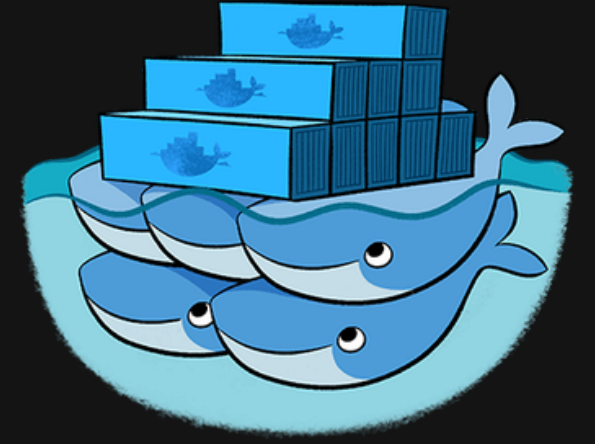
Volumen (Volume)

Permite almacenar datos de forma persistente, evitando que estos se pierdan cuando un contenedor se elimina o se recrea. Es especialmente útil para bases de datos.

```
> docker volume ls
> docker volume create datos
```

Docker

Componentes



Red (Network)

Permite que los contenedores se comuniquen entre sí. Por ejemplo, un contenedor con una API puede comunicarse con otro que contiene una base de datos.

```
> docker network ls docker
> docker network create mi-red
```

Dockerfile

Archivo de texto que contiene las instrucciones para construir una imagen Docker, indicando la imagen base, archivos, dependencias y comando de ejecución.

```
> docker build -t mi-app .
```

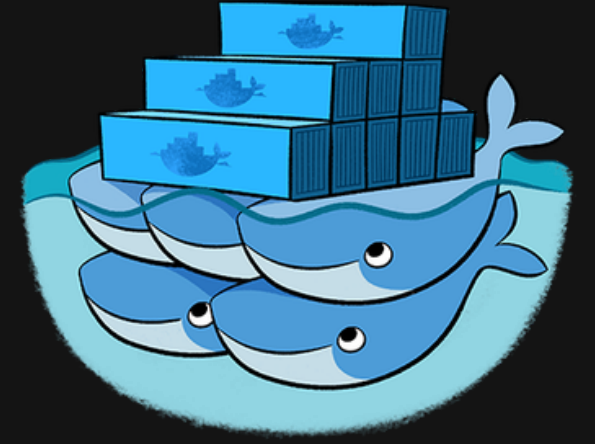
Registry

Repositorio donde se almacenan y distribuyen imágenes.

```
> docker pull imagen
```

Docker

Algunos comandos



```
> docker images
> docker pull <imagen>
> docker build -t <nombre> .
> docker rmi <imagen>
> docker run <imagen>
> docker ps
> docker ps -a
> docker start <container>
> docker stop <container>
> docker restart <container>
> docker rm <container>
```

```
# Listar imágenes
# Descargar una imagen
# Crear una imagen
# Eliminar una imagen
# Crear y ejecutar un contenedor
# Ver contenedores activos
# Ver todos los contenedores
# Iniciar un contenedor
# Detener un contenedor
# Reiniciar un contenedor
# Eliminar un contenedor
```

Docker Compose



¿Qué es?

Docker Compose es una herramienta de Docker que permite definir, configurar y ejecutar múltiples contenedores como una sola aplicación.

En lugar de ejecutar cada contenedor manualmente, se utiliza un archivo `compose.yaml` donde se especifican los servicios, redes, volúmenes, puertos y variables de entorno.

¿Para qué se utiliza?

- Gestionar varios contenedores de una aplicación.
- Configurar la comunicación entre servicios.
- Administrar volúmenes y almacenamiento persistente.
- Definir variables de entorno y configuraciones.
- Levantar toda la aplicación con un solo comando.
- Facilitar ambientes de desarrollo y pruebas reproducibles.

Docker Compose

Comandos



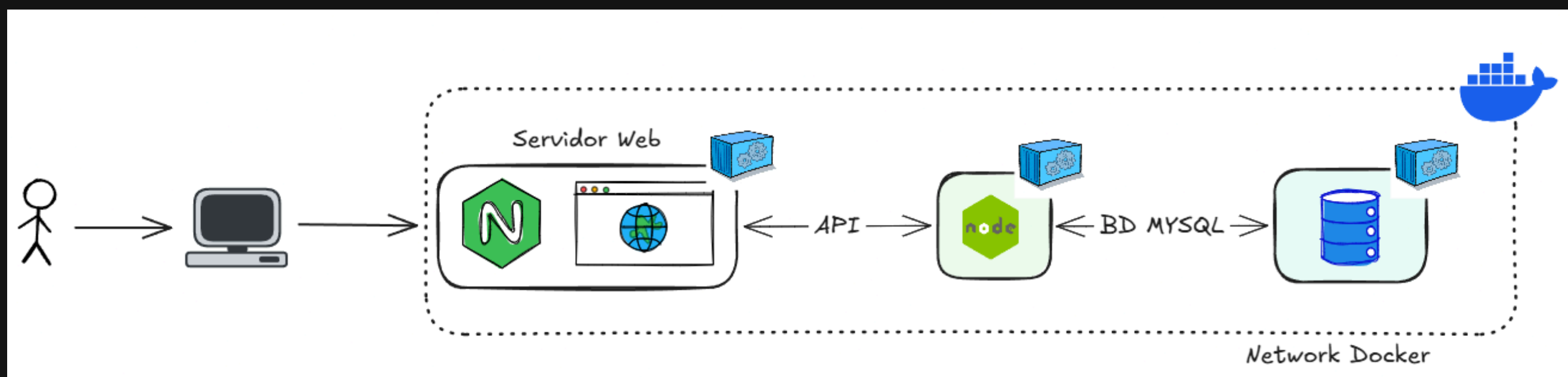
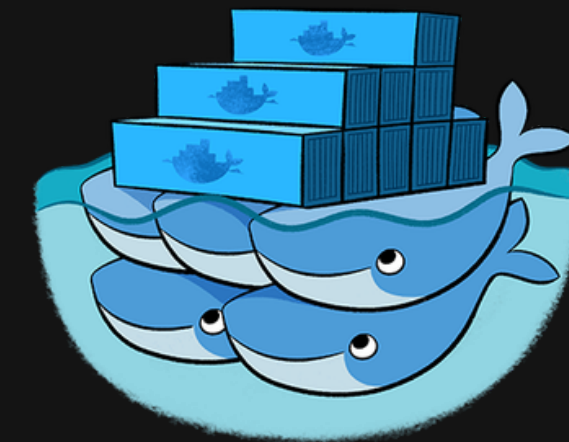
```
> docker compose up           # Crear e iniciar servicios
> docker compose up -d        # Ejecutar en segundo plano
> docker compose down         # Detener y eliminar servicios
> docker compose ps           # Ver servicios
> docker compose logs          # Ver logs
> docker compose build         # Construir imágenes
```

Pongámoslo a prueba!



Docker

Ejemplo



¿Dudas?



Repositorio con el ejemplo

